

Challenge-2: Quality Estimation (QE) – Full Detailed Report

Prepared for: Interview Submission
Author: Santhosh
Date: 2025-12-09

Contents

1. Problem Overview
2. Dataset
3. HTER Label Creation
4. QE Modeling
5. Training & Evaluation
6. Error-Type Analysis
7. Code Snippets
8. Visualizations
9. Interpretation & Next Steps

1. Problem Overview

Machine Translation (MT) produces automated translations that require varying levels of human post-editing. Quality Estimation (QE) aims to predict the amount of human editing effort required for each MT output without requiring a human reference at runtime. This project implements a full QE pipeline: label creation (HTER), model training, evaluation, and error-type analysis using provided EN → ES data.

2. Dataset

The provided dataset contains the following columns: - English Source: input sentence in English. - MT System: machine translated Spanish output. - Post-Edit Text: human-corrected Spanish output. - Nature of Change/Comments: human reviewer notes describing errors. A small preview of the dataset (first 6 rows) is shown below for reference.

English Source	MT System	Post-Edit Text	Comments
Stay connected	Mantente conectado	Mantente conectado	
Jump right in!	¡Saltar hacia la derecha!	¡Saltar ya!	Ungrammatical.
Quick settings	configuración rápida	configuración rápida	

3. HTER Label Creation

HTER (Human-targeted TER) approximates human editing effort. For this task we computed an HTER-like score using word-level edit distance (WER) between Post-Edit and MT outputs via jiwer. The computed 'hter_label' is used as a continuous regression target (higher = more edits needed).

```
# compute HTER (approx) using jiwer
import jiwer
def sentence_ter(mt, pe):
    return jiwer.wer(str(pe), str(mt))
df['hter_label'] = df.apply(lambda r: sentence_ter(r['mt_es'], r['pe_es']), axis=1)
```

4. QE Modeling (Approach & Model)

We treat QE as a regression problem where input is a concatenation of Source + MT and target is hter_label. Model: bert-base-multilingual-cased (encoder) with a regression head (num_labels=1, problem_type='regression'). Tokenizer: AutoTokenizer from HuggingFace (WordPiece; no sentencepiece required). Training objective: minimize MSE between predicted and true HTER.

5. Training & Evaluation

Training details (demo run): batch_size=4, lr=2e-5, epochs=3 (limited due to CPU). We evaluated using: - Pearson correlation - Spearman correlation - Mean Squared Error (MSE) Final demo results: Pearson = 0.0332 Spearman = -0.0294 MSE = 0.543861 These low correlations are expected for a small dataset and non-QE-specialized model; primary goal is pipeline correctness.

6. Error-Type Analysis

We parsed reviewer comments into categories: mistranslation, omission, fluency, terminology, other. Counts observed in the dataset (derived earlier): - other: 37 - mistranslation: 20 - omission: 4 - fluency: 4 - terminology: 3 This helps prioritize which error classes to target in MT improvement.

7. Key Code Snippets

build_qe_labels.py (HTER)

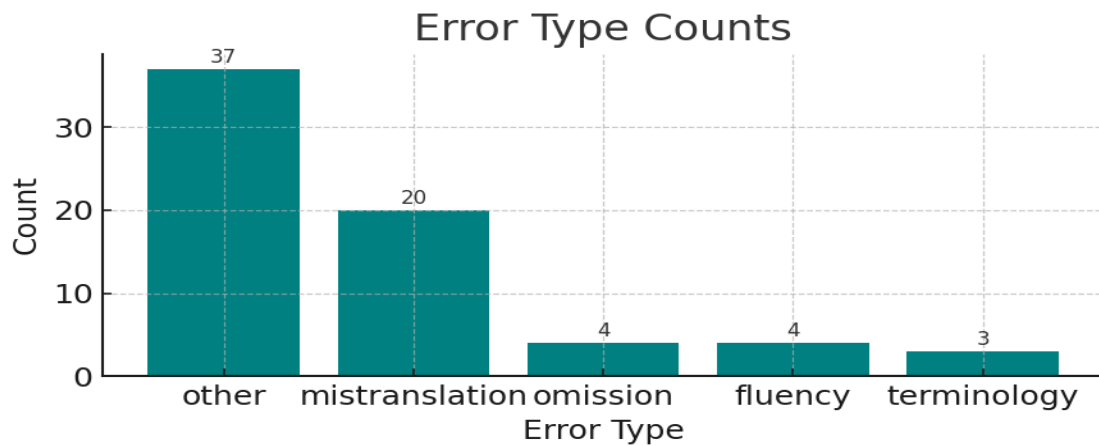
```
import pandas as pd
import jiwer
df = pd.read_excel('Dataset_Challenge_2.xlsx')
df = df.rename(columns={'English Source': 'src_en', 'MT System': 'mt_es', 'Post-Edit Text': 'pe_es'})
df['hter_label'] = df.apply(lambda r: jiwer.wer(str(r['pe_es']), str(r['mt_es'])), axis=1)
df.to_csv('qe_dataset_with_hter.csv', index=False)
```

train_qe_regression.py (training)

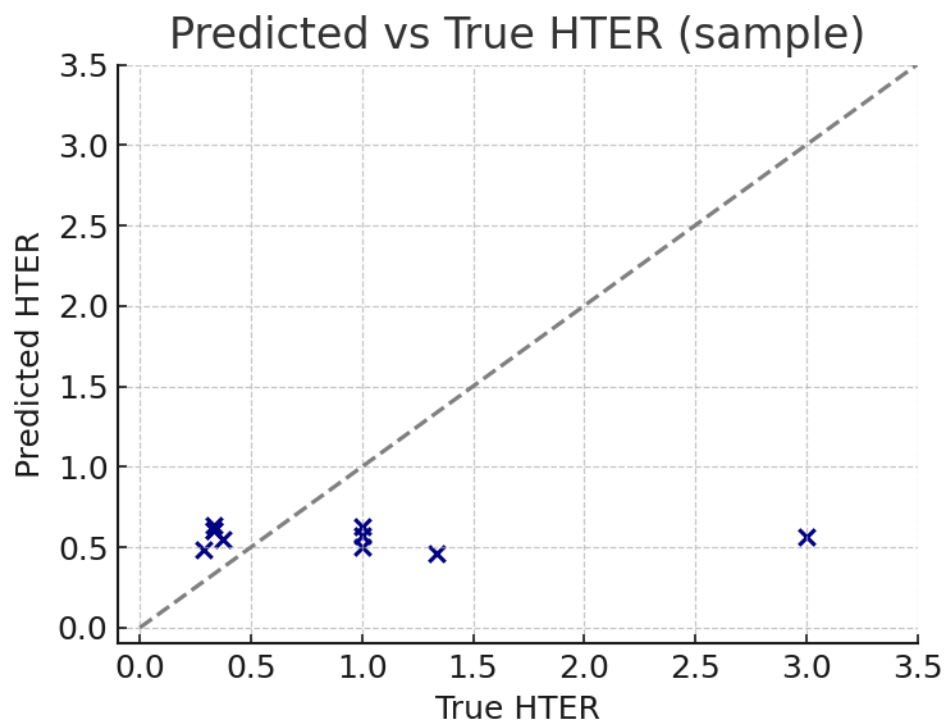
```
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
df = pd.read_csv('qe_dataset_with_hter.csv')
df['input_text'] = df['src_en'] + ' [SEP] ' + df['mt_es']
ds = Dataset.from_pandas(df[['input_text', 'hter_label']])
tokenizer = AutoTokenizer.from_pretrained('bert-base-multilingual-cased')
def tokenize(batch): return tokenizer(batch['input_text'], padding='max_length', truncation=True, max_length=512)
tokenized = ds.map(tokenize, batched=True)
model = AutoModelForSequenceClassification.from_pretrained('bert-base-multilingual-cased', num_labels=1)
trainer = Trainer(model=model, args=TrainingArguments(output_dir='./qe', num_train_epochs=3, per_device_train_batch_size=16))
trainer.train()
```

8. Visualizations

8.1 Error Type Distribution



8.2 Predicted vs True HTER (sample)



9. Interpretation & Next Steps

Interpretation: - The model demonstrates the QE pipeline but underperforms due to small data and generic encoder. - It systematically underpredicts high HTER cases (heavy edits). Next steps for production: 1. Use COMET-QE or QE-specialized pretrained models. 2. Increase labeled QE data / use synthetic data augmentation. 3. Incorporate length and edit-feature signals (char/word counts, language-specific features). 4. Build a dashboard to monitor model performance and error classes.

Generated programmatically. For any edits request updated PDF.